UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

BugBoard26

Documentazione BugBoard26

(BugBoard26 Documentation)

Antonio Scafaro	*N86005219*
Alessio Paduano	*N86005142*
Luigi Pio Scalice	*N86005197*

Corso di Ingegneria del Software

A.A. 2025/2026

Indice

1 Analisi dei requisiti **1**

1.1 Introduzione . . . 1

1.1.1 Scopo . . . 1

1.1.2 Ambito del prodotto . . . 1

1.1.3 Definizioni, acronimi e abbreviazioni . . . 3

1.1.4 Riferimenti . . . 3

1.2 Panoramica del documento . . . 4

1.3 Glossario . . . 5

1.3.1 Termini del dominio . . . 5

1.3.2 Acronimi e abbreviazioni . . . 8

1.4 Casi d'Uso . . . 9

1.4.1 Diagramma dei casi d'uso . . . 9

1.4.2 Lista dei casi d'uso . . . 11

1.5 Individuazione e caratterizzazione degli utenti del sistema . . . 14

1.5.1 Utente . . . 14

1.5.2 Utente_Autenticato . . . 15

1.6 Amministratore . . . 17

1.6.1 Descrizione . . . 17

1.6.2 Obiettivi . . . 17

1.6.3 Responsabilità e permessi . . . 17

1.7 Riepilogo Permessi per Profilo . . . 19

1.8 Requisiti Specifici . . . 20

1.8.1 Requisiti di Sistema . . . 20

1.8.2 Requisiti Funzionali . . . 22

1.8.3 Requisiti Non Funzionali . . . 49

1.8.4 Riepilogo . . . 54

1.9 Descrizione Dettagliata Casi d'Uso (Template Cockburn) . . . 55

1.9.1 Caso d'Uso #5: Segnalazione Issue . . . 55

1.10 Mockup delle Interfacce Utente . . . 59

1.10.1 Mockup Creazione Issue 59
1.10.2 Descrizione Schermate 60
1.10.3 M3, M4, M5 - Messaggi Errore 60
1.10.4 M6 - Board Kanban 61
1.10.5 Principi di Design 61

2 Design del sistema **62**

2.1 System Design 62
2.1.1 Criteri di progettazione 62
2.1.2 Motivazioni architetturali 64

1 Analisi dei requisiti

1.1 Introduzione

Questo documento rappresenta la Specifica dei Requisiti Software (SRS) per il sistema BugBoard26, un'applicazione per la gestione collaborativa di issue in progetti software. Il documento è conforme allo standard IEEE 830-1998.

1.1.1 Scopo

Lo scopo di questo documento è fornire una specifica completa, accurata e verificabile dei requisiti funzionali e non funzionali del sistema BugBoard26. Questo SRS è destinato ai seguenti stakeholder:

- **Team di sviluppo**: per comprendere nel dettaglio le funzionalità da implementare e i vincoli da rispettare.
- **Project manager**: per pianificare attività, risorse e tempistiche di sviluppo.
- **Quality Assurance**: per definire criteri di accettazione e test plan.
- **Committente**: per validare che i requisiti soddisfino le esigenze di business.

Il documento costituisce la base contrattuale tra il team di sviluppo e il committente (SoftEngUniNA), definendo in modo non ambiguo ciò che il sistema deve fare e come deve comportarsi.

1.1.2 Ambito del prodotto

Nome del prodotto: BugBoard26

Descrizione generale: BugBoard26 è una piattaforma distribuita per la gestione collaborativa di issue (segnalazioni) in progetti software. Il sistema consente di:

- Organizzare il lavoro in progetti distinti, ciascuno con il proprio team
- Segnalare issue (bug, feature, domande, documentazione) all'interno dei progetti autorizzati

- Monitorare lo stato di avanzamento delle issue attraverso workflow configurabili
- Assegnare issue a membri del team del progetto
- Collaborare tramite commenti e discussioni
- Generare report mensili per progetto per analizzare performance del team

Gestione Progetti e Permessi:

Gli amministratori hanno controllo completo su:

- Creazione, modifica ed eliminazione progetti
- Assegnazione utenti ai progetti (chi può accedere a cosa)
- Gestione membri del team per ciascun progetto

Gli utenti standard possono:

- Visualizzare solo i progetti a cui sono stati assegnati dagli amministratori
- Creare e gestire issue solo all'interno dei progetti a cui hanno accesso
- Lavorare sulle issue che gli vengono assegnate dagli amministratori

Benefici attesi:

- Miglioramento della tracciabilità dei problemi e delle richieste
- Riduzione del tempo medio di risoluzione grazie a una migliore organizzazione
- Aumento della trasparenza e della collaborazione all'interno del team
- Supporto decisionale attraverso analisi quantitative delle performance

Obiettivi principali:

- Fornire un sistema performante (tempo di risposta $\leq$ 3000ms per operazioni comuni)
- Garantire la sicurezza dei dati sensibili attraverso autenticazione robusta
- Assicurare l'usabilità con un'interfaccia intuitiva che richieda training minimo
- Facilitare la manutenibilità attraverso un'architettura distribuita e modulare

Cosa NON è incluso nell'ambito:

- Integrazione con sistemi di versionamento del codice (Git, SVN)
- Funzionalità di continuous integration/deployment

1.1.3 Definizioni, acronimi e abbreviazioni

Per una trattazione completa dei termini specifici del dominio, degli acronimi e delle abbreviazioni utilizzati in questo documento, si rimanda alla Sezione 1.3 (Glossario).

Alcuni termini chiave includono:

- **Issue**: segnalazione relativa al progetto (bug, feature, question, documentation)
- **BE**: Back-end, componente server-side del sistema
- **FE**: Front-end, componente client-side del sistema
- **API**: Application Programming Interface, interfaccia REST per la comunicazione BE-FE

1.1.4 Riferimenti

Questo documento fa riferimento ai seguenti standard, specifiche e documenti tecnici:

Rif.	Descrizione
RIF-1	*Progetto-SWE-2025-2026 - canale 2_v1.pdf*. Specifica del committente con requisiti di alto livello e vincoli del progetto.
RIF-2	*IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*. Standard di riferimento per la struttura e il contenuto di documenti SRS.
RIF-3	*Cockburn, A. (2000). Writing Effective Use Cases*. Metodologia per la descrizione testuale strutturata di casi d'uso.

1.2 Panoramica del documento

Il presente documento è organizzato come segue:

- **Sezione 1.1 – Introduzione**: fornisce una panoramica del documento, descrive lo scopo del sistema, definisce gli acronimi e elenca i riferimenti normativi e bibliografici.
- **Sezione 1.3 – Glossario**: contiene le definizioni di tutti i termini tecnici, gli acronimi e i riferimenti utilizzati nel documento per evitare ambiguità.
- **Sezione 1.4 – Casi d'Uso**: presenta il diagramma UML dei casi d'uso e la lista completa delle funzionalità del sistema, modellando le interazioni tra gli attori e il sistema.
- **Sezione 1.5 – Caratterizzazione Utenti**: identifica e descrive in dettaglio i profili degli utenti del sistema (Utente, Utente Autenticato, Amministratore), specificandone obiettivi, responsabilità e permessi.
- **Sezione 1.8 – Requisiti Specifici**: costituisce il cuore tecnico del documento e include:
  - **Requisiti di Sistema**: vincoli architetturali e di sistema imposti dal committente
  - **Requisiti Funzionali**: descrizione dettagliata di tutte le funzionalità assegnate
  - **Requisiti Non Funzionali**: requisiti di qualità misurabili e verificabili (usabilità, performance, sicurezza, manutenibilità, affidabilità)
- **Sezione 1.9 – Descrizione Dettagliata Casi d'Uso**: fornisce la formalizzazione testuale strutturata secondo il template di Cockburn per un caso d'uso significativo.
- **Sezione 1.10 – Mockup**: presenta la prototipazione visuale delle interfacce utente per i casi d'uso formalizzati, collegata agli step delle tabelle Cockburn.

Questo documento deve essere letto in modo sequenziale al primo approccio. Le sezioni successive possono essere consultate in modo non lineare come riferimento durante le fasi di progettazione, implementazione e testing.

1.3 Glossario

1.3.1 Termini del dominio

Glossario dei Termini

Termine	Descrizione
Account	Insieme di informazioni e credenziali (email, password) che identificano univocamente un utente e ne regolano l'accesso al sistema.
Amministratore	Utente con ruolo privilegiato che possiede tutti i permessi di un utente autenticato, più funzionalità aggiuntive di gestione del sistema (creazione utenze, assegnazione issue, archiviazione, accesso ai report).
Archiviazione	Operazione riservata agli Amministratori che sposta una issue in una sezione di archivio, rimuovendola dalle liste principali ma mantenendone l'accessibilità da una vista dedicata.
Assegnazione	Operazione con cui un Amministratore associa una issue a un membro specifico del team, rendendolo responsabile della sua gestione.
Autenticazione	Processo di verifica dell'identità di un utente tramite credenziali (email e password) per consentire l'accesso alle funzionalità del sistema.
Bug	Tipo di issue che descrive un errore, difetto o malfunzionamento del software rispetto al comportamento atteso.
Commento	Messaggio testuale lasciato da un utente autenticato all'interno della sezione di discussione di una issue specifica.

Dashboard	Pagina principale visualizzata dopo il login, che fornisce una panoramica delle issue e delle attività rilevanti per l'utente.
Documentation	Tipo di issue che segnala problemi, mancanze o necessità di chiarimenti nella documentazione del progetto.
Etichetta (Label)	Marcatore testuale personalizzabile associabile a una issue per facilitarne la categorizzazione, il filtraggio e la ricerca (es. "frontend", "urgente", "sicurezza").
Feature	Tipo di issue che rappresenta la richiesta di una nuova funzionalità o un miglioramento del sistema esistente.
Filtro	Criterio di selezione applicabile alla lista delle issue per visualizzare solo quelle che soddisfano specifiche condizioni (es. tipo, stato, priorità).
Issue	Segnalazione relativa al progetto; entità fondamentale del sistema. Può essere di quattro tipi: Bug, Feature, Question, Documentation. Ogni issue ha attributi obbligatori (titolo, descrizione, tipo, priorità, stato) e opzionali (etichette, immagini, assegnatario).
Metrica	Misura quantitativa aggregata utilizzata nei report mensili per valutare performance e attività del team (es. numero di issue aperte, tempo medio di risoluzione).
Notifica	Messaggio informativo inviato automaticamente dal sistema a un utente in seguito a eventi specifici (es. assegnazione di una issue).
Ordinamento	Operazione che modifica la sequenza di visualizzazione delle issue secondo un criterio specificato (es. data di creazione, priorità) e una direzione (crescente o decrescente).

Priorità	Attributo opzionale che definisce il livello di urgenza di una issue. Valori ammessi: Nessuna, Bassa, Media, Alta, Critica.
Progetto	Spazio di lavoro organizzativo che raggruppa un insieme di issue relative a uno specifico prodotto software.
Question	Tipo di issue utilizzato per porre domande o richiedere chiarimenti sul progetto.
Report Mensile	Documento riepilogativo generato automaticamente dal sistema, contenente metriche aggregate sull'attività del team in un intervallo temporale di un mese. Accessibile solo agli Amministratori.
Ruolo	Attributo associato a un account utente che determina l'insieme di permessi e funzionalità accessibili. Valori possibili: Utente (standard) o Amministratore.
Stato	Fase corrente nel ciclo di vita di una issue. Stato iniziale: "ToDo". Altri stati possibili includono: In Lavorazione, Risolta, Chiusa, Archiviata.
Team	Insieme degli utenti che collaborano al progetto utilizzando la piattaforma BugBoard26.
ToDo	Stato iniziale di una issue appena creata, che indica che non è ancora stata presa in carico da alcun membro del team.
Utente	Individuo che interagisce con il sistema. Prima dell'autenticazione, può accedere solo alla schermata di login.
Utente Autenticato	Utente che ha completato con successo il processo di autenticazione e ha accesso alle funzionalità di base del sistema (creazione, visualizzazione e modifica issue, commenti).

1.3.2 Acronimi e abbreviazioni

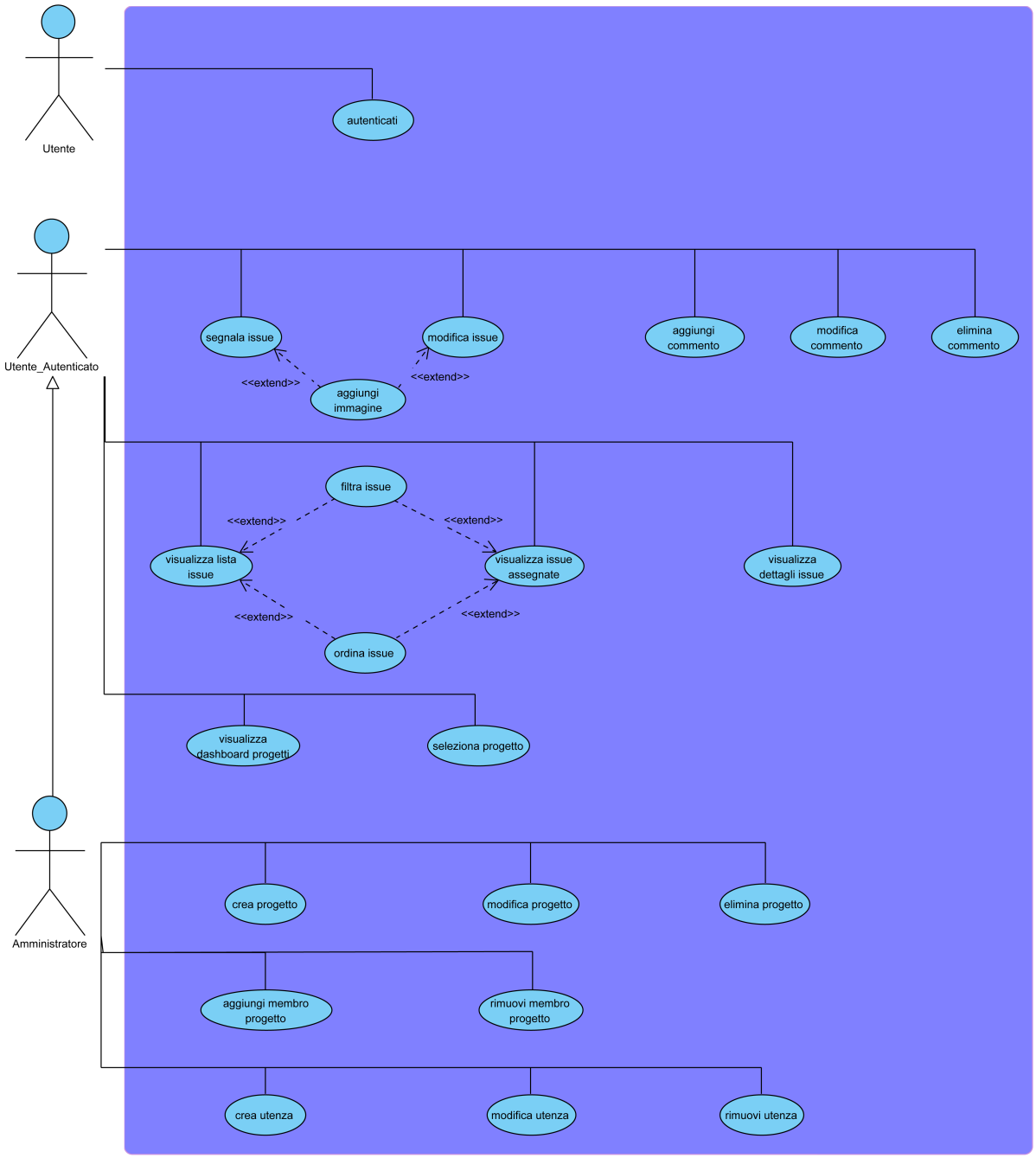
Acronimo	Significato
API	*Application Programming Interface* – Interfaccia di programmazione che definisce le modalità di interazione tra componenti software.
BE	*Back-end* – Componente server-side del sistema, responsabile della logica di business, gestione dati e persistenza.
FE	*Front-end* – Componente client-side del sistema, responsabile dell'interfaccia utente e dell'interazione con l'utente finale.
HTTPS	*HyperText Transfer Protocol Secure* – Protocollo di comunicazione sicuro per il trasferimento di dati su Internet.
IEEE	*Institute of Electrical and Electronics Engineers* – Organizzazione professionale che pubblica standard internazionali.
JSON	*JavaScript Object Notation* – Formato leggero per lo scambio di dati strutturati.
JWT	*JSON Web Token* – Standard per la creazione di token di accesso che consentono l'autenticazione stateless.
NFR	*Non-Functional Requirement* – Requisito non funzionale, che specifica criteri di qualità del sistema.
REST	*Representational State Transfer* – Stile architetturale per la progettazione di API basate su HTTP.
SRS	*Software Requirements Specification* – Documento di specifica dei requisiti software.
UC	*Use Case* – Caso d'uso, descrizione di un'interazione tra un attore e il sistema per raggiungere un obiettivo specifico.
UML	*Unified Modeling Language* – Linguaggio di modellazione visuale standard per sistemi software.
URL	*Uniform Resource Locator* – Indirizzo che specifica la posizione di una risorsa su Internet.

1.4 Casi d'Uso

Questa sezione presenta la modellazione del sistema attraverso casi d'uso UML.

1.4.1 Diagramma dei casi d'uso

La Figura sottostante presenta il diagramma UML dei casi d'uso completo.



Note sul diagramma:

- L'attore *Utente* rappresenta qualunque individuo che accede al sistema prima dell'autenticazione.
- L'attore *Utente_Autenticato* eredita implicitamente i permessi di *Utente* e può accedere alle funzionalità di gestione issue.
- L'attore *Amministratore* eredita tutti i permessi di *Utente_Autenticato* e dispone di funzionalità privilegiate (gestione utenze, assegnazione, archiviazione, report).
- Le relazioni `«extend»` indicano funzionalità opzionali che possono essere eseguite in aggiunta al caso d'uso principale.

1.4.2 Lista dei casi d'uso

Sono stati individuati i seguenti casi d'uso per il sistema BugBoard26:

1. **UC-01: Autenticazione**
   Attore primario: Utente
   Descrizione: l'utente inserisce email e password per accedere al sistema.

2. **UC-02: Visualizza Dashboard Progetti**
   Attore primario: Utente Autenticato, Amministratore
   Descrizione: l'utente visualizza la lista dei progetti a cui ha accesso assegnato dall'amministratore, con statistiche e attività recenti per ciascun progetto.

3. **UC-03: Seleziona Progetto**
   Attore primario: Utente Autenticato, Amministratore
   Descrizione: l'utente seleziona uno dei progetti a cui ha accesso per visualizzarne le issue.

4. **UC-04: Crea Progetto**
   Attore primario: Amministratore
   Descrizione: l'amministratore crea un nuovo progetto specificando nome, descrizione e membri iniziali del team.

5. **UC-05: Modifica Progetto**
   Attore primario: Amministratore
   Descrizione: l'amministratore modifica nome, descrizione o altre proprietà di un progetto esistente.

6. **UC-06: Elimina Progetto**
   Attore primario: Amministratore
   Descrizione: l'amministratore elimina un progetto e tutte le sue issue associate.

7. **UC-07: Aggiungi Membri Progetto**
   Attore primario: Amministratore
   Descrizione: l'amministratore aggiunge utenti dal team di un progetto, determinando chi può accedere a quel progetto.

8. **UC-08: Rimuovi Membri Progetto**
   Attore primario: Amministratore
   Descrizione: l'amministratore rimuove utenti dal team di un progetto, determinando chi non può più accedere a quel progetto.

9. **UC-09: Segnala Issue**
   Attore primario: Utente Autenticato, Amministratore
   Descrizione: l'utente crea una nuova issue all'interno del progetto correntemente selezionato.

10. **UC-10: Visualizza Lista Issue**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente visualizza tutte le issue del progetto corrente.

11. **UC-11: Visualizza Dettagli Issue**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente visualizza il dettaglio completo di una issue del progetto corrente.

12. **UC-12: Modifica Issue**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente modifica attributi di una issue esistente nel progetto.

13. **UC-13: Aggiungi Etichette**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente associa etichette a una issue per categorizzazione.

14. **UC-14: Aggiungi Immagine**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente allega immagini a una issue.

15. **UC-15: Filtra Issue**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente applica filtri per visualizzare sottoinsiemi di issue del progetto.

16. **UC-16: Ordina Issue**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente ordina la lista issue secondo criteri specificati.

17. **UC-17: Visualizza Issue Assegnate**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente visualizza solo le issue del progetto corrente che gli sono state assegnate dall'amministratore.

18. **UC-18: Aggiungi Commento**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente aggiunge commenti testuali a una issue.

19. **UC-19: Elimina Commento**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente elimina i propri commenti.

20. **UC-20: Visualizza Notifiche**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente visualizza notifiche ricevute (assegnazioni, menzioni, aggiornamenti).

21. **UC-21: Prendi Visione Notifica**
    Attore primario: Utente Autenticato, Amministratore
    Descrizione: l'utente marca le notifiche come lette.

22. **UC-22: Creazione Utenze**
    Attore primario: Amministratore
    Descrizione: l'amministratore crea nuovi account utente.

23. **UC-23: Modifica Utenze**
    Attore primario: Amministratore
    Descrizione: l'amministratore modifica dati di account esistenti.

24. **UC-24: Rimozione Utenze**
    Attore primario: Amministratore
    Descrizione: l'amministratore elimina account utente.

25. **UC-25: Assegna Issue**
    Attore primario: Amministratore
    Descrizione: l'amministratore assegna una issue a un membro del team del progetto.

26. **UC-26: Archivia Issue**
    Attore primario: Amministratore
    Descrizione: l'amministratore archivia una issue mantenendone la tracciabilità.

27. **UC-27: Visualizza Report Mensile**
    Attore primario: Amministratore
    Descrizione: l'amministratore visualizza report mensile con metriche del progetto selezionato.

1.5 Individuazione e caratterizzazione degli utenti del sistema

Il sistema BugBoard26 è progettato per supportare tre distinte tipologie di utenti, ciascuna con obiettivi, responsabilità e permessi specifici. Questa sezione fornisce una caratterizzazione dettagliata di ogni profilo utente.

1.5.1 Utente

1.5.1.1 Descrizione

L'attore *Utente* rappresenta qualunque individuo che accede all'applicazione BugBoard26 prima di aver completato il processo di autenticazione. Questo profilo corrisponde allo stato iniziale di interazione con il sistema, limitato alla sola schermata di login.

1.5.1.2 Obiettivi

- Autenticarsi al sistema utilizzando credenziali valide (email e password)
- Accedere alle funzionalità della piattaforma secondo il proprio ruolo

1.5.1.3 Responsabilità e permessi

- **Visualizzare** la schermata di autenticazione
- **Inserire** email e password nei campi del form di login
- **Inviare** una richiesta di autenticazione al sistema
- **Ricevere** messaggi di errore in caso di credenziali non valide o account bloccato

Limitazioni: questo profilo non ha accesso ad alcuna funzionalità operativa del sistema fino al completamento dell'autenticazione.

1.5.2 Utente_Autenticato

1.5.2.1 Descrizione

L'**Utente Autenticato** è un membro del team che ha effettuato correttamente il login e ha accesso ai **progetti a cui è stato assegnato da un amministratore**. Questo profilo rappresenta lo sviluppatore, tester o membro del team che lavora attivamente su issue all'interno dei progetti autorizzati.

1.5.2.2 Obiettivi

- Visualizzare i progetti a cui ha accesso assegnato dall'amministratore
- Lavorare sulle issue dei progetti autorizzati (creazione, modifica, commenti)
- Collaborare con il team attraverso commenti e aggiornamenti sulle issue
- Monitorare lo stato delle issue assegnate personalmente
- Ricevere e gestire notifiche relative ai progetti di appartenenza

1.5.2.3 Responsabilità e permessi

L'Utente_Autenticato dispone dei seguenti permessi:

Gestione Progetti

- **Può**: visualizzare solo i progetti a cui è stato aggiunto da un amministratore, selezionare un progetto per accedere alle sue issue
- **Non può**: creare nuovi progetti, eliminare progetti, modificare progetti, scegliere autonomamente a quali progetti partecipare, vedere progetti di cui non è membro

Gestione Issue

- **Creare** nuove issue, all'interno dei progetti a cui appartiene, specificando:
  - Titolo (*obbligatorio*)
  - Descrizione (*obbligatoria*)
  - Tipo (*obbligatorio*: Bug, Feature, Question, Documentation)

  - Priorità (*opzionale*: Nessuna (Default), Bassa, Media, Alta, Critica)
  - Etichette personalizzate (*opzionale*, array di stringhe)
  - Immagini allegate (*opzionale*)

- **Visualizzare** la lista completa delle issue del progetto selezionato
- **Filtrare** le issue per tipo, stato, priorità, etichetta, assegnatario, intervallo di data creazione
- **Ordinare** le issue per data di creazione, priorità, stato (crescente/decrescente)
- **Visualizzare** il dettaglio completo di una singola issue con tutti i suoi attributi
- **Modificare** le issue esistenti (aggiornamento di titolo, descrizione, tipo, priorità, etichette), solo per i progetti di appartenenza
- **Associare** etichette personalizzate a una issue per facilitarne la categorizzazione

Collaborazione

- **Aggiungere** commenti testuali a qualunque issue
- **Eliminare** i propri commenti
- **Leggere** i commenti lasciati da altri utenti

1.5.2.4 Vista Personalizzata

- **Visualizzare** una vista dedicata delle issue assegnate al proprio account
- **Ricevere** notifiche quando una issue viene assegnata al proprio account
- **Visualizzare** l'elenco delle notifiche ricevute
- **Confermare** la presa visione delle notifiche

Limitazioni: questo profilo non può creare o modificare utenze, non può assegnare issue ad altri utenti, non può archiviare issue e non ha accesso ai report amministrativi.

1.6 Amministratore

1.6.1 Descrizione

L'*Amministratore* è un utente con privilegi elevati, responsabile della gestione delle utenze, della supervisione del flusso di lavoro e del monitoraggio delle performance del team. Questo ruolo eredita completamente tutti i permessi dell'Utente_Autenticato e dispone di funzionalità amministrative aggiuntive.

1.6.2 Obiettivi

- **Gestire** gli accessi al sistema creando, modificando e rimuovendo account utente
- **Garantire** la presa in carico tempestiva delle issue critiche attraverso l'assegnazione esplicita ai membri del team
- **Supervisionare** l'andamento del progetto analizzando report mensili con metriche quantitative
- **Mantenere** ordinate le viste principali archiviando le issue non più rilevanti ma preservandone la tracciabilità
- **Monitorare** le performance individuali e del team per supportare decisioni strategiche

1.6.3 Responsabilità e permessi

L'Amministratore dispone di:

1.6.3.1 Permessi ereditati

- **Tutti** i permessi e le responsabilità dell'Utente_Autenticato (creazione issue, visualizzazione, filtri, commenti, notifiche)

1.6.3.2 Gestione Utenze

- **Creare** nuove utenze specificando:
  - Nome (obbligatorio)
  - Cognome (obbligatorio)
  - Email (obbligatoria, univoca nel sistema)
  - Password (obbligatoria, soggetta a policy di complessità)
  - Ruolo (obbligatorio: Utente o Amministratore)
- **Modificare** i dati anagrafici e le credenziali di utenze esistenti
- **Eliminare** utenze dal sistema previa conferma con inserimento della propria password amministrativa

1.6.3.3 Gestione Avanzata Issue

- **Assegnare** una issue a un membro specifico del team, rendendolo responsabile della sua gestione
- **Archiviare** una issue, spostandola dalle liste principali a una sezione di archivio dedicata (la issue rimane accessibile ma non appare nelle viste standard)

1.6.3.4 Analisi e Reportistica

- **Visualizzare** report mensili contenenti metriche aggregate sull'attività del team:
  - Numero totale di issue aperte nel periodo
  - Numero totale di issue gestite/risolte nel periodo
  - Tempo medio di risoluzione (aggregato e per singolo utente)
  - Distribuzione delle issue per tipo e priorità
  - Performance individuali di ciascun membro del team

Nota: il sistema è fornito con un account amministratore predefinito con credenziali di default, che permette di effettuare il primo accesso e creare ulteriori utenze.

1.7 Riepilogo Permessi per Profilo

La seguente tabella riassume i permessi differenziati per ciascun profilo utente:

Funzionalità	Utente	Utente Aut.	Admin
Autenticazione (login)	✓	✓	✓
Visualizza dashboard progetti	✗	✓(solo assegnati)	✓(tutti)
Crea progetto	✗	✗	✓
Modifica progetto	✗	✗	✓
Elimina progetto	✗	✗	✓
Gestisci membri progetto	✗	✗	✓
Seleziona progetto	✗	✓(solo assegnati)	✓(tutti)
Crea issue	✗	✓(progetti autorizzati)	✓
Visualizza issue	✗	✓(progetti autorizzati)	✓(tutti)
Modifica issue	✗	✓(progetti autorizzati)	✓(tutti)
Assegna issue	✗	✗	✓
Archivia issue	✗	✗	✓
Aggiungi commento	✗	✓	✓
Elimina proprio commento	✗	✓	✓
Filtra/ordina issue	✗	✓	✓
Aggiungi etichette/immagini	✗	✓	✓
Visualizza notifiche	✗	✓	✓
Crea utenza	✗	✗	✓
Modifica utenza	✗	✗	✓
Elimina utenza	✗	✗	✓
Visualizza report	✗	✗	✓

Legenda:

- ✓ = Permesso concesso
- ✗ = Permesso negato
- "solo assegnati" = può accedere solo a progetti in cui appare in `Project_Member`
- "progetti autorizzati" = operazioni limitate ai progetti di appartenenza

1.8 Requisiti Specifici

1.8.1 Requisiti di Sistema

Questa sezione dettaglia i requisiti specifici che il sistema BugBoard26 deve soddisfare. I requisiti sono definiti da una prospettiva esterna, descrivendo le interazioni del sistema con i suoi utenti e con altri sistemi.

1.8.1.1 Requisiti delle Interfacce Esterne

Interfaccia Utente (User Interface)

Il sistema dovrà fornire un'interfaccia utente grafica (GUI) web-based, accessibile tramite i browser moderni. L'interfaccia dovrà essere responsive, garantendo l'usabilità su schermi di diverse dimensioni, dai dispositivi mobili (minimo 320px di larghezza) ai desktop. Tutte le funzionalità esposte agli utenti dovranno essere accessibili tramite tale interfaccia.

Interfaccia Software (Software Interface)

Il sistema dovrà esporre un'interfaccia di programmazione (API) basata sullo stile architetturale REST per consentire la comunicazione tra il componente front-end e il back-end. Queste API dovranno utilizzare il formato JSON per lo scambio di dati e seguire le convenzioni standard del protocollo HTTP per le operazioni (GET, POST, PUT, DELETE). Questo garantirà l'interoperabilità e la possibilità di sviluppare client alternativi in futuro.

Interfaccia di Comunicazione (Communications Interface)

Tutta la comunicazione tra il client e il server che trasporta dati sensibili (incluse le credenziali di autenticazione e i dati delle issue) dovrà avvenire tramite protocolli di comunicazione sicuri (HTTPS).

1.8.1.2 Requisiti Prestazionali

Il sistema dovrà rispettare i requisiti di performance specificati nella sezione dei Requisiti Non Funzionali (REQ-NFR-01), garantendo tempi di risposta rapidi per le operazioni interattive.

1.8.1.3 Requisiti del Database Logico

Il sistema dovrà garantire la persistenza, la consistenza e l'integrità dei dati. Dovrà gestire le seguenti entità logiche principali e le loro relazioni: Utenti, Progetti, Issue, Commenti, Etichette, Allegati e le associazioni tra di esse (es. Utenti membri di Progetti, Issue appartenenti a Progetti). Il sistema dovrà garantire che le operazioni sul database siano transazionali per prevenire la corruzione dei dati in caso di fallimenti.

1.8.2 Requisiti Funzionali

Questa sottosezione descrive in dettaglio tutte le funzionalità del sistema BugBoard26, specificando input, elaborazione e output secondo lo standard IEEE 830-1998. Il sistema è organizzato gerarchicamente: gli **Amministratori** creano **Progetti** e assegnano **Utenti** ad essi; all'interno di ogni progetto, i membri autorizzati gestiscono **Issue**.

REQ-FUN-01: Autenticazione

Introduzione

Il sistema deve fornire un meccanismo di autenticazione sicuro per accesso alla piattaforma.

Input

- Email (obbligatorio, formato valido)
- Password (obbligatorio, min 8 caratteri)

Elaborazione

1. Valida formato email e presenza campi
2. Cerca account nel database
3. Se valida: crea sessione
4. Se non valida: incrementa contatore tentativi

Output

Successo: Reindirizzamento a dashboard progetti
Errore: "Credenziali non valide", "Account bloccato", "Campi obbligatori"

REQ-FUN-02: Visualizza Dashboard Progetti

Introduzione

Dopo l'autenticazione, l'utente visualizza la dashboard con i **progetti a cui ha accesso** assegnati dall'amministratore.

Precondizioni

Utente autenticato.

Input

Caricamento automatico post-login.

Elaborazione

1. Recupera l'utente autenticato dal database
2. Recupera i progetti legati all'utente

Output

Dashboard con:

- Card progetti: nome, descrizione, statistiche
- Pulsante "Accedi" per selezionare un progetto
- Se nessun progetto: "Nessun progetto disponibile."
- Se admin: pulsante "+ Crea Nuovo Progetto"

REQ-FUN-03: Seleziona Progetto

1.8.2.1 Introduzione

L'utente seleziona un progetto per accedere alle sue issue.

1.8.2.2 Precondizioni

Utente autenticato con almeno un progetto assegnato.

1.8.2.3 Input

Click su progetto dalla dashboard.

1.8.2.4 Elaborazione

1. Verifica che l'utente appartenga al progetto selezionato
2. Imposta il progetto corrente nella sessione
3. Recupera le issue del progetto
4. Reindirizza alla lista issue

1.8.2.5 Output

Successo: Entra nel contesto del progetto, header mostra nome progetto, visualizza lista issue
Errore: "Accesso negato. Non fai parte di questo progetto."

REQ-FUN-04: Crea Progetto (solo Admin)

1.8.2.6 Introduzione

Solo gli amministratori possono creare nuovi progetti.

1.8.2.7 Precondizioni

Utente con ruolo Amministratore.

1.8.2.8 Input

- Nome progetto (obbligatorio, univoco)
- Descrizione (opzionale)
- Membri iniziali (obbligatorio, multi-selezione utenti, con almeno un membro)

1.8.2.9 Elaborazione

1. Verifica ruolo amministratore
2. Valida unicità nome progetto
3. Crea nuovo progetto nel database
4. Associa i membri selezionati al progetto
5. Aggiunge automaticamente l'amministratore creatore come membro

1.8.2.10 Output

Successo: "Progetto '[Nome]' creato con successo"
Errore: "Nome già esistente", "Permessi insufficienti"

REQ-FUN-05: Modifica Progetto (solo Admin)

1.8.2.11 Introduzione

Gli amministratori possono modificare nome e descrizione di progetti esistenti.

1.8.2.12 Precondizioni

Amministratore, progetto esistente.

1.8.2.13 Input

Nuovo nome (se modificato deve essere univoco), nuova descrizione.

1.8.2.14 Elaborazione

1. Verifica ruolo amministratore
2. Se nome modificato: verifica unicità
3. Aggiorna i dati del progetto nel database

1.8.2.15 Output

Successo: "Progetto aggiornato con successo"
Errore: "Nome già in uso", "Permessi insufficienti"

REQ-FUN-06: Elimina Progetto (solo Admin)

1.8.2.16 Introduzione

Gli amministratori possono eliminare progetti con tutte le issue associate.

1.8.2.17 Precondizioni

Amministratore, progetto esistente.

1.8.2.18 Input

Selezione progetto, conferma con password amministratore.

1.8.2.19 Elaborazione

1. Mostra dialog conferma: "Eliminare progetto? Tutte le issue saranno eliminate. Operazione irreversibile."
2. Verifica password amministratore
3. Elimina tutte le issue, commenti, etichette e allegati del progetto
4. Rimuove associazioni membri-progetto
5. Elimina il progetto dal database

1.8.2.20 Output

Successo: "Progetto '[Nome]' e tutte le sue issue eliminate"
Errore: "Password non valida"

REQ-FUN-07: Aggiungi Membri Progetto (solo Admin)

1.8.2.21 Introduzione

Gli amministratori decidono quali utenti possono accedere a ciascun progetto.

1.8.2.22 Precondizioni

Amministratore, progetto esistente.

1.8.2.23 Input

Selezione utenti da aggiungere al progetto.

1.8.2.24 Elaborazione

Aggiunta membro:

1. Verifica che l'utente non sia già membro del progetto
2. Associa l'utente al progetto nel database
3. Notifica l'utente dell'aggiunta

1.8.2.25 Output

Successo: "Utente [Nome] aggiunto al progetto"
Errore: "Utente già membro".

REQ-FUN-08: Rimuovi Membri Progetto (solo Admin)

1.8.2.26 Introduzione

Gli amministratori decidono quali utenti devono essere rimossi da un determinato progetto.

1.8.2.27 Precondizioni

Amministratore, progetto esistente, utente_autenticato membro del progetto

1.8.2.28 Input

Selezione utenti da rimuovere dal progetto.

1.8.2.29 Elaborazione

Rimozione membro:

1. Verifica che il progetto abbia almeno un altro membro
2. Rimuove le assegnazioni issue dell'utente nel progetto
3. Rimuove l'associazione utente-progetto dal database
4. Notifica l'utente della rimozione

1.8.2.30 Output

Successo: "Utente [Nome] rimosso dal progetto"
Errore: "Impossibile rimuovere ultimo membro"

REQ-FUN-09: Segnala Issue

1.8.2.31 Introduzione

Gli utenti creano nuove issue all'interno del **progetto correntemente selezionato**.

1.8.2.32 Precondizioni

Utente autenticato, progetto selezionato, utente membro del progetto.

1.8.2.33 Input

- Titolo (obbligatorio)
- Descrizione (obbligatoria)
- Tipo (obbligatorio: Bug, Feature, Question, Documentation)
- Priorità (opzionale: Nessuna, Bassa, Media, Alta, Critica)
- Etichette (opzionale)
- Immagini (opzionale)

1.8.2.34 Elaborazione

1. Verifica che l'utente appartenga al progetto corrente
2. Valida tutti i campi obbligatori e i vincoli di formato
3. Se presenti immagini: carica i file su storage
4. Se priorità non specificata: imposta a "Nessuna"
5. Crea la nuova issue associata al progetto corrente con stato "ToDo"
6. Se priorità è "Critica": notifica gli amministratori del progetto

1.8.2.35 Output

Successo: "Issue #[ID] creata con successo", reindirizzamento a dettaglio issue
Errore: "Campo obbligatorio mancante", "Immagine troppo grande", "Formato non supportato"

REQ-FUN-10: Visualizza Lista Issue

1.8.2.36 Introduzione

Gli utenti visualizzano tutte le issue del **progetto corrente**.

1.8.2.37 Precondizioni

Utente autenticato, progetto selezionato, membro del progetto.

1.8.2.38 Input

Accesso a sezione "Issue" o automatico dopo selezione progetto.

1.8.2.39 Elaborazione

1. Verifica appartenenza al progetto corrente
2. Recupera tutte le issue del progetto non archiviate
3. Per ogni issue recupera: ID, titolo, tipo, priorità, stato, autore, assegnatario, data
4. Applica eventuali filtri e ordinamenti salvati in sessione

1.8.2.40 Output

Lista issue con: tabella o card, icona tipo, titolo cliccabile, priorità (badge colorato), stato, assegnatario, pulsante "Crea Nuova Issue".

REQ-FUN-11: Visualizza Dettagli Issue

1.8.2.41 Introduzione

Gli utenti visualizzano il dettaglio completo di una issue del progetto corrente.

1.8.2.42 Precondizioni

Utente autenticato, membro del progetto della issue.

1.8.2.43 Input

Click su issue dalla lista.

1.8.2.44 Elaborazione

1. Verifica che la issue appartenga al progetto corrente
2. Verifica che l'utente sia membro del progetto
3. Recupera tutti i dettagli della issue: informazioni base, commenti, etichette, immagini

1.8.2.45 Output

Pagina dettaglio / Modale con: header (ID, titolo, tipo, priorità, stato), informazioni (autore, data creazione, assegnatario), descrizione formattata, etichette, immagini (galleria), sezione commenti, pulsanti "Modifica" e "Archivia" (se amministratore). **Errore**: "Issue non trovata" o "Accesso negato" se progetto non autorizzato.

REQ-FUN-12: Modifica Issue

1.8.2.46 Introduzione

Gli utenti possono modificare gli attributi delle issue del progetto corrente.

1.8.2.47 Precondizioni

Utente autenticato, membro del progetto, issue non archiviata.

1.8.2.48 Input

Modifiche a: titolo, descrizione, tipo, priorità, stato, etichette.

1.8.2.49 Elaborazione

1. Verifica appartenenza al progetto
2. Valida tutti i campi modificati
3. Confronta i valori con quelli precedenti
4. Aggiorna la issue nel database
5. Se priorità diventa "Critica": notifica amministratori
6. Se stato diventa "Risolta": notifica autore della issue

1.8.2.50 Output

Successo: "Issue aggiornata con successo"

REQ-FUN-13: Aggiungi Etichette

1.8.2.51 Introduzione

Gli utenti possono associare etichette personalizzate alle issue per facilitarne la categorizzazione.

1.8.2.52 Precondizioni

Utente autenticato, membro del progetto.

1.8.2.53 Input

Testo etichetta.

1.8.2.54 Elaborazione

1. Normalizza il testo dell'etichetta
2. Verifica che non sia già associata alla issue
3. Verifica che il totale etichette non superi 10
4. Associa l'etichetta alla issue

1.8.2.55 Output

Aggiunta: Tag colorato visibile immediatamente.
Rimozione: Click su X del tag rimuove l'etichetta dalla issue.

REQ-FUN-14: Aggiungi Immagine

1.8.2.56 Introduzione

Gli utenti possono allegare immagini alle issue per fornire documentazione visuale.

1.8.2.57 Input

File immagine PNG o JPG.

1.8.2.58 Elaborazione

1. Valida formato file e dimensione
2. Genera nome file univoco
3. Carica il file su storage
4. Associa l'immagine alla issue nel database

1.8.2.59 Output

Successo: "Immagine caricata con successo", pulsante "Rimuovi".
Errore: "Formato non supportato. Usa PNG o JPG"

REQ-FUN-15: Filtra Issue

1.8.2.60 Introduzione

Gli utenti possono applicare filtri per visualizzare sottoinsiemi specifici di issue del **progetto corrente**.

1.8.2.61 Input

Selezione criteri di filtro: Tipo, Stato, Priorità, Etichetta, Assegnatario, Autore, Data creazione (range).

1.8.2.62 Elaborazione

1. Costruisce i criteri di filtro selezionati dall'utente
2. Recupera dal database solo le issue del progetto corrente che soddisfano tutti i criteri
3. Salva i filtri applicati nella sessione per persistenza

1.8.2.63 Output

Lista issue aggiornata con solo gli elementi filtrati, badge "X filtri attivi".

REQ-FUN-16: Ordina Issue

1.8.2.64 Introduzione

Gli utenti possono ordinare la lista issue secondo criteri specificati.

1.8.2.65 Input

Campo di ordinamento (Data creazione, Priorità, Stato, Titolo) e direzione (Crescente o Decrescente).

1.8.2.66 Elaborazione

1. Applica l'ordinamento richiesto alla lista issue
2. Per Priorità usa ordine: Critica, Alta, Media, Bassa
3. Per Stato usa ordine: ToDo, In Lavorazione, Risolta, Chiusa
4. Salva la preferenza di ordinamento nella sessione

1.8.2.67 Output

Lista issue riordinata secondo il criterio selezionato, indicatore visuale (freccia) sulla colonna ordinata, persistenza durante la navigazione.

REQ-FUN-17: Visualizza Issue Assegnate

1.8.2.68 Introduzione

Gli utenti visualizzano una vista dedicata con solo le issue del **progetto corrente** assegnate a loro.

1.8.2.69 Input

Click su "Le Mie Issue" dal menu.

1.8.2.70 Elaborazione

1. Recupera tutte le issue del progetto corrente assegnate all'utente
2. Esclude le issue archiviate

1.8.2.71 Output

Lista filtrata con solo le issue assegnate all'utente, organizzazione per stato (ToDo, In Lavorazione, Risolta, Chiusa), possibilità di cambiare stato direttamente.

REQ-FUN-18: Aggiungi Commento

1.8.2.72 Introduzione

Gli utenti possono aggiungere commenti testuali alle issue per discussione e collaborazione.

1.8.2.73 Input

Testo commento (obbligatorio).

1.8.2.74 Elaborazione

1. Valida lunghezza testo
2. Sanitizza l'input
3. Crea nuovo commento associato alla issue con timestamp e autore
4. Se il commento menziona altri utenti (@username): invia notifiche
5. Aggiorna timestamp ultima modifica della issue

1.8.2.75 Output

Commento appare immediatamente nella sezione commenti con avatar autore, nome, timestamp, testo formattato, pulsante "Elimina" (visibile solo all'autore), toast "Commento aggiunto"

REQ-FUN-19: Elimina Commento

1.8.2.76 Introduzione

Gli utenti possono eliminare i propri commenti; gli amministratori possono eliminare qualsiasi commento.

1.8.2.77 Precondizioni

Autore del commento oppure amministratore.

1.8.2.78 Elaborazione

1. Mostra dialog conferma: "Eliminare commento? Operazione irreversibile."
2. Se confermato: elimina il commento dal database o lo marca come eliminato

1.8.2.79 Output

Commento scompare dalla lista o mostra "[Commento eliminato]", toast "Commento eliminato"

REQ-FUN-20: Visualizza Notifiche

1.8.2.80 Introduzione

Gli utenti visualizzano notifiche relative ai progetti a cui hanno accesso.

1.8.2.81 Input

Click sull'icona campanella nell'header.

1.8.2.82 Elaborazione

1. Recupera tutte le notifiche dell'utente
2. Filtra per progetti a cui l'utente ha accesso
3. Ordina per timestamp decrescente (più recenti in alto)
4. Separa notifiche lette da non lette

1.8.2.83 Output

Dropdown o pagina notifiche con: lista notifiche (icona tipo, testo, timestamp relativo "2 ore fa"), badge rosso con numero notifiche non lette, evidenziazione notifiche non lette, pulsante "Segna tutte come lette"
Tipi notifiche: Issue assegnata, Issue Critica creata (solo admin), Issue risolta (per autore), Menzione in commento, Nuovo commento su issue seguita.

REQ-FUN-21: Prendi Visione Notifica

1.8.2.84 Introduzione

Gli utenti possono marcare le notifiche come lette.

1.8.2.85 Elaborazione

1. Marca la notifica come letta nel database
2. Aggiunge timestamp di lettura
3. Decrementa il contatore badge notifiche non lette

1.8.2.86 Output

Notifica cambia aspetto (sfondo normale invece di evidenziato), badge contatore aggiornato, se cliccata reindirizza alla issue o commento relativo.

REQ-FUN-22: Creazione Utenze (solo Admin)

1.8.2.87 Introduzione

Gli amministratori possono creare nuovi account utente.

1.8.2.88 Input

Nome, Cognome, Email (univoca), Password (min 8 caratteri con requisiti di complessità), Ruolo (Utente o Amministratore).

1.8.2.89 Elaborazione

1. Verifica ruolo amministratore
2. Valida tutti i campi e unicità email
3. Valida complessità password (almeno 1 maiuscola, 1 minuscola, 1 numero, 1 carattere speciale)
4. Crea nuovo account utente nel database

1.8.2.90 Output

Successo: "Utente [Nome Cognome] creato con successo", reindirizzamento a lista utenti
Errore: "Email già registrata", "Password non rispetta requisiti di complessità"

REQ-FUN-23: Modifica Utenze (solo Admin)

1.8.2.91 Introduzione

Gli amministratori possono modificare i dati degli account esistenti.

1.8.2.92 Input

Selezione utente, modifiche a: Nome, Cognome, Email, Password (opzionale), Ruolo.

1.8.2.93 Elaborazione

1. Verifica che l'utente da modificare esista
2. Previene auto-modifica del ruolo amministratore corrente
3. Se email modificata: verifica unicità
4. Se password modificata: genera nuovo hash e invalida sessioni attive dell'utente
5. Aggiorna i dati dell'account nel database

1.8.2.94 Output

Successo: "Utente aggiornato con successo"
Errore: "Email già in uso", "Non puoi modificare il tuo ruolo amministratore"

REQ-FUN-24: Rimozione Utenze (solo Admin)

1.8.2.95 Introduzione

Gli amministratori possono eliminare account utente previa conferma.

1.8.2.96 Input

Selezione utente da eliminare, password amministratore per conferma.

1.8.2.97 Elaborazione

1. Dialog: "Eliminare utente? Operazione irreversibile. Le issue create rimarranno ma l'autore sarà marcato come 'eliminato'."
2. Verifica password amministratore
3. Previene auto-eliminazione
4. Invalida tutte le sessioni attive dell'utente
5. Marca le issue create dall'utente come "autore eliminato"
6. Rimuove l'utente da tutte le assegnazioni issue
7. Rimuove l'utente da tutti i progetti
8. Elimina l'account dal database

1.8.2.98 Output

Successo: "Utente [Nome Cognome] eliminato"
Errore: "Password amministratore non valida", "Non puoi eliminare il tuo account"

REQ-FUN-25: Assegna Issue (solo Admin)

1.8.2.99 Introduzione

Gli amministratori possono assegnare issue a membri del team del progetto.

1.8.2.100 Precondizioni

Amministratore, issue e utente target esistenti, utente target membro del progetto della issue.

1.8.2.101 Input

Issue da assegnare, selezione utente assegnatario dalla lista membri del progetto.

1.8.2.102 Elaborazione

1. Verifica che l'utente target sia membro del progetto
2. Aggiorna l'assegnatario della issue
3. Se la issue era già assegnata: notifica il precedente assegnatario della riassegnazione
4. Crea notifica per il nuovo assegnatario

1.8.2.103 Output

"Issue #[ID] assegnata a [Nome]", issue appare in "Le Mie Issue" dell'assegnatario, notifica inviata.

Errore: "Utente non è membro del progetto"

REQ-FUN-26: Archivia Issue (solo Admin)

1.8.2.104 Introduzione

Gli amministratori possono archiviare issue mantenendone la tracciabilità.

1.8.2.105 Precondizioni

Amministratore, issue esistente non già archiviata.

1.8.2.106 Elaborazione

1. Dialog: "Archiviare issue #[ID]? Non sarà più visibile nelle liste ma rimarrà accessibile dall'archivio."
2. Se confermato: marca la issue come archiviata con timestamp e amministratore che ha eseguito l'operazione
3. La issue viene esclusa dalle liste standard
4. Rimane accessibile dalla vista "Archivio" (solo amministratori)

1.8.2.107 Output

"Issue #[ID] archiviata", scompare dalla lista standard, pulsante "Ripristina" disponibile.

Ripristino: L'amministratore può ripristinare la issue dalla vista Archivio.

REQ-FUN-27: Visualizza Report Mensile (solo Admin)

1.8.2.108 Introduzione

Gli amministratori visualizzano report mensile del **progetto selezionato** con metriche aggregate.

1.8.2.109 Precondizioni

Amministratore, progetto selezionato, dati di almeno un mese di attività.

1.8.2.110 Input

Selezione mese e anno tramite date picker (default: mese corrente), click su "Genera Report".

1.8.2.111 Elaborazione

Il sistema calcola le seguenti metriche per il progetto corrente nel periodo selezionato:

Metriche Globali del Progetto:

- Numero issue create e risolte nel mese
- Numero issue ancora aperte
- Distribuzione percentuale per tipo e priorità
- Tempo medio di risoluzione in ore

Metriche per Utente (membri del progetto):

- Per ogni membro: issue create, assegnate, risolte
- Tempo medio risoluzione personale
- Percentuale efficienza (risolte su assegnate)

Allerte: lista issue critiche non risolte da più di 7 giorni

1.8.2.112 Output

Dashboard report con: card KPI con numeri principali, grafici per distribuzioni, tabelle ripelogative per utente.

Errore: "Permessi insufficienti", "Nessun dato disponibile per il periodo selezionato"

1.8.3 Requisiti Non Funzionali

Questa sottosezione descrive i requisiti non funzionali del sistema BugBoard26 secondo lo standard IEEE 830-1998 e il modello di qualità ISO/IEC 25002. I requisiti non funzionali specificano attributi di qualità del sistema, vincoli sulle modalità implementative e proprietà che il sistema deve soddisfare indipendentemente dalle sue funzionalità specifiche.

I requisiti non funzionali sono spesso più critici dei requisiti funzionali: mentre gli utenti possono trovare soluzioni alternative a una funzionalità sub-ottimale, il mancato rispetto di un requisito non funzionale può rendere il sistema completamente inutilizzabile.

I requisiti sono organizzati secondo la classificazione:

- **Product Requirements**: requisiti che specificano caratteristiche intrinseche del prodotto software (performance, usabilità, affidabilità, sicurezza)
- **Organizational Requirements**: requisiti derivanti da politiche e procedure dell'organizzazione (linguaggi obbligatori, standard da rispettare, processi operativi)
- **External Requirements**: requisiti derivanti da fonti esterne (conformità normativa, requisiti etici, requisiti legali)

1.8.3.1 Product Requirements

I Product Requirements definiscono le caratteristiche di qualità intrinseche del sistema BugBoard26, basate sul modello ISO/IEC 25002.

REQ-NFR-01: Performance

Descrizione

Il sistema deve garantire tempi di risposta rapidi per le operazioni più comuni.

Requisiti

- Le operazioni di visualizzazione (dashboard, lista issue, dettagli) devono completarsi entro **1 secondo**
- Le operazioni di creazione e modifica (nuova issue, commento) devono completarsi entro **2 secondi**
- La generazione di report mensili deve completarsi entro **5 secondi**
- Il caricamento iniziale dell'applicazione deve completarsi entro **3 secondi**

REQ-NFR-02: Usabilità

Descrizione

Il sistema deve essere facile da usare e intuitivo per ridurre il tempo di apprendimento.

Requisiti

- L'interfaccia deve essere chiara e auto-esplicativa senza necessità di manuale utente
- Le operazioni più comuni (crea issue, visualizza lista, commenta) devono essere accessibili in massimo **3 click**
- I form devono mostrare messaggi di errore chiari quando l'input non è valido
- Le azioni distruttive (elimina progetto, elimina utente) devono richiedere conferma esplicita

REQ-NFR-03: Affidabilità

1.8.3.2 Descrizione

Il sistema deve funzionare in modo stabile senza crash o perdita di dati.

1.8.3.3 Requisiti

- Il sistema non deve presentare crash durante l'uso normale
- Eventuali errori del server devono essere gestiti mostrando messaggi chiari all'utente invece di bloccare l'applicazione
- I dati inseriti dall'utente non devono mai essere persi (ogni operazione deve essere persistita nel database)

REQ-NFR-04: Sicurezza

1.8.3.4 Descrizione

Il sistema deve proteggere i dati degli utenti e impedire accessi non autorizzati.

1.8.3.5 Requisiti

- Le password devono essere salvate come hash, mai in chiaro
- Le sessioni devono essere gestite con durata limitata
- Ogni richiesta API deve verificare che l'utente sia autenticato e autorizzato
- Gli utenti possono accedere solo ai progetti di cui fanno parte
- Solo gli amministratori possono creare progetti, gestire utenti e assegnare issue
- La comunicazione in produzione deve avvenire su HTTPS
- Gli input utente devono essere validati per prevenire SQL Injection

REQ-NFR-05: Manutenibilità

1.8.3.6 Descrizione

Il codice deve essere organizzato in modo chiaro per facilitare manutenzione e modifiche future.

1.8.3.7 Requisiti

- Il backend deve essere organizzato in layer separati: Controller, Service
- Il frontend deve utilizzare componenti React riusabili e ben organizzati
- Il codice deve seguire convenzioni di naming consistenti
- Devono essere presenti commenti per spiegare logiche complesse
- Il progetto deve avere un README con istruzioni per setup e deployment
- Il codice deve essere formattato automaticamente (Prettier per TypeScript, formatter Java)

REQ-NFR-06: Compatibilità

1.8.3.8 Descrizione

Il sistema deve funzionare correttamente sui browser più comuni.

1.8.3.9 Requisiti

- L'applicazione deve funzionare su Chrome, Firefox, Edge e Safari versioni recenti
- L'interfaccia deve essere responsive e utilizzabile su qualsiasi schermo da 320px in su
- Le API REST devono restituire JSON strutturato con status code HTTP corretti

REQ-NFR-07: Conformità Normativa

1.8.3.10 Descrizione

Il sistema deve rispettare le normative di base sulla protezione dei dati.

1.8.3.11 Requisiti

- Il sistema deve raccogliere solo i dati necessari (nome, cognome, email, password)
- I dati personali devono essere protetti e accessibili solo agli utenti autorizzati
- Gli amministratori possono eliminare account utente quando richiesto
- Le dipendenze open-source devono avere licenze compatibili (MIT, Apache 2.0)

1.8.4 Riepilogo

La seguente tabella riassume i requisiti non funzionali con priorità:

ID	Requisito	Priorità
REQ-NFR-01	Performance	Alta
REQ-NFR-02	Usabilità	Alta
REQ-NFR-03	Affidabilità	Media
REQ-NFR-04	Sicurezza	Critica
REQ-NFR-05	Manutenibilità	Media
REQ-NFR-06	Compatibilità	Media
REQ-NFR-07	Conformità Normativa	Alta

1.9 Descrizione Dettagliata Casi d'Uso (Template Cockburn)

Questa sezione fornisce la formalizzazione testuale strutturata di un caso d'uso significativo secondo il template di Alistair Cockburn. Il caso d'uso selezionatp è rappresentativo delle funzionalità core del sistema ed è stato scelto per dimostrare la completezza della specifica.

1.9.1 Caso d'Uso #5: Segnalazione Issue

Use Case #5	**Segnalazione Issue**
Goal in Context	Un utente autenticato oppure un amministratore vogliono segnalare una nuova issue per tracciare un problema, richiedere una funzionalità, porre una domanda o segnalare un problema di documentazione.
Preconditions	
Success End Conditions	La issue viene creata con successo e aggiunta alla lista delle issue del progetto con stato "ToDo". L'utente può visualizzare la issue appena creata e riceve conferma dell'operazione.
Failed End Conditions	Nessun cambiamento viene effettuato al sistema. La issue non viene creata se i dati obbligatori sono incompleti o non validi.
Primary Actor	Utente Autenticato oppure Amministratore
Trigger	L'utente clicca sul pulsante "Crea Nuova Issue" o "Segnala Issue" dalla dashboard o dalla lista issues.

Main Scenario

Step	User Action	System Response
1	Clicca su "Segnala nuova issue" dalla dashboard	
2		Mostra il form di creazione issue vuoto (M1)
3	Compila i campi obbligatori: • Titolo, • Descrizione, • Tipo, Opzionalmente, sceglie priorità e/o aggiunge etichette e/o immagini Clicca sul pulsante "Crea Issue"	
4		Reindirizza alla pagina di vista Kanban (M6) e mostra il messaggio di successo "Issue creata con successo" (M2)

Estensione A: Campo Titolo mancante

Step	User Action	System Response
3.1a	Compila tutti i campi tranne il **Titolo** e clicca "Crea Issue"	
3.2a		Mostra messaggio di errore (M3)
3.3a	Clicca "OK" o chiude il messaggio di errore	
3.4a		Il form rimane visualizzato con i dati già inseriti preservati e ritorna allo step 3 dello scenario principale

Estensione B: Campo Descrizione mancante

Step	User Action	System Response
3.1b	Compila tutti i campi tranne la **Descrizione** e clicca "Crea Issue"	
3.2b		Mostra messaggio di errore (M4)
3.3b	Clicca "OK" o chiude il messaggio di errore	
3.4b		Il form rimane visualizzato con i dati già inseriti preservati e ritorna allo step 3 dello scenario principale

Estensione C: Campo Tipo mancante

Step	User Action	System Response
3.1c	Compila tutti i campi tranne il **Tipo** e clicca "Crea Issue"	
3.2c		Mostra messaggio di errore (M5)
3.3c	Clicca "OK" o chiude il messaggio di errore	
3.4c		Il form rimane visualizzato con i dati già inseriti preservati e ritorna allo step 3 dello scenario principale

Estensione D: Annullamento operazione

Step	User Action	System Response
3.1d	Durante la compilazione del form, clicca su "Annulla" oppure sulla "X" per chiudere	
3.2d		Chiude il form senza salvare, reindirizza alla pagina precedente con vista Kanban (M6)

1.10 Mockup delle Interfacce Utente

Questa sezione presenta i mockup delle interfacce principali del sistema BugBoard26. I mockup mostrano il layout e il flusso di interazione delle schermate.

1.10.1 Mockup Creazione Issue

La Figura sottostante mostra il flusso completo di creazione di una nuova issue, includendo il form principale e i messaggi di feedback (successo ed errori).

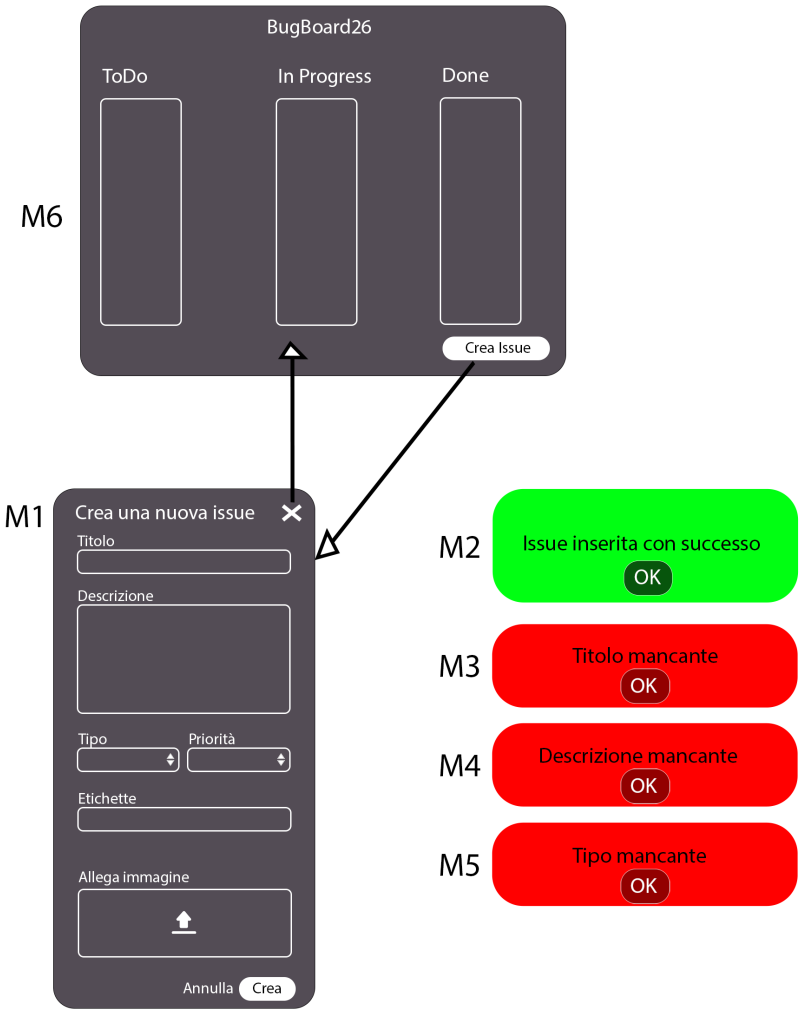


Figura 1: Mockup flusso creazione issue. La figura mostra: **M1** - Form creazione con campi Titolo, Descrizione, Tipo, Priorità, Etichette e Upload immagini; **M2** - Messaggio successo "Issue inserita con successo"; **M3** - Errore "Titolo mancante"; **M4** - Errore "Descrizione mancante"; **M5** - Errore "Tipo mancante"; **M6** - Vista board Kanban con colonne ToDo, In Progress, Done.

1.10.2 Descrizione Schermate

1.10.2.1 M1 - Form Creazione Issue

Form principale per creare nuove issue nel progetto corrente.

1.10.2.2 Elementi

- Header "Crea una nuova issue" con pulsante chiusura (X)
- Campo Titolo (obbligatorio, input testo singola riga)
- Campo Descrizione (obbligatorio, textarea multiriga)
- Dropdown Tipo (obbligatorio: Bug, Feature, Question, Documentation)
- Dropdown Priorità (opzionale: Nessuna, Bassa, Media, Alta, Critica)
- Campo Etichette (opzionale)
- Area upload immagini
- Pulsanti "Annulla" e "Crea" in basso

1.10.2.3 M2 - Conferma Successo

Toast notification che appare dopo la creazione con successo della issue.

Contenuto

- Testo "Issue inserita con successo"
- Pulsante "OK"
- Scompare automaticamente dopo 3 secondi o al click su OK

1.10.3 M3, M4, M5 - Messaggi Errore

Alert che appaiono quando mancano campi obbligatori.

Contenuto

- M3: "Titolo mancante"
- M4: "Descrizione mancante"
- M5: "Tipo mancante"
- Pulsante "OK" per chiudere
- Il campo mancante viene evidenziato in rosso nel form

1.10.4 M6 - Board Kanban

Vista alternativa delle issue organizzate in colonne per stato.

Elementi

- Header "BugBoard26"
- Almeno tre colonne: "ToDo", "In Progress", "Done"
- Pulsante "Crea Issue" in basso a destra

Note

La board Kanban è una vista alternativa alla lista tabellare, in cui sono presenti colonne corrispettive agli stati con all'interno le varie issue. Le issue possono essere spostate tra colonne con drag&drop per cambiare lo stato in modo efficiente.

1.10.5 Principi di Design

I mockup seguono questi principi:

- **Semplicità**: layout pulito con elementi essenziali
- **Feedback immediato**: messaggi di successo/errore chiari
- **Validazione**: campi obbligatori evidenziati quando mancanti

2 Design del sistema

2.1 System Design

Il sistema BugBoard26 è stato progettato seguendo un'architettura distribuita a tre livelli (three-tier), in cui ciascun componente (front-end, back-end e database) è sviluppato, distribuito e scalato in modo indipendente. Questa scelta architetturale è motivata dalla necessità di garantire **separazione delle responsabilità**, **manutenibilità** e **scalabilità** del sistema nel tempo.

2.1.1 Criteri di progettazione

Le decisioni architetturali e implementative sono state guidate dai seguenti criteri di design:

- **Separazione delle responsabilità (Separation of Concerns)**: ogni componente del sistema ha un ruolo ben definito. Il front-end si occupa esclusivamente della presentazione e dell'interazione con l'utente; il back-end gestisce la logica di business, la validazione e l'accesso ai dati; il database garantisce la persistenza. All'interno del back-end, la logica è ulteriormente stratificata in livelli distinti (Controller, Service, Repository), ciascuno con responsabilità specifiche e non sovrapposte.
- **Architettura a livelli (Layered Architecture)**: il back-end adotta un'architettura a livelli conforme ai principi di Spring Boot, in cui ogni strato comunica esclusivamente con quello immediatamente adiacente:
  - **Controller**: riceve le richieste HTTP, effettua la deserializzazione dei parametri e delega l'elaborazione al livello Service. Non contiene logica di business.
  - **Service**: implementa la logica applicativa, coordina le operazioni tra più repository e gestisce le regole di business (es. controllo permessi, notifiche, validazioni trasversali).
  - **Repository**: astrae l'accesso al database tramite il pattern Repository di Spring Data JPA, esponendo metodi di query dichiarativi.
  - **Entity**: modella le tabelle del database come classi Java annotate con JPA, definendo relazioni, vincoli e mapping object-relational.
- **Data Transfer Objects (DTO)**: la comunicazione tra i livelli e verso il client avviene tramite oggetti DTO dedicati, distinti dalle entity di persistenza. Questo pattern

garantisce il **disaccoppiamento** tra la rappresentazione interna dei dati e la loro esposizione tramite API, consentendo di controllare quali informazioni vengono esposte, evitare dipendenze circolari nella serializzazione e favorire l'evoluzione indipendente del modello dati e delle interfacce REST.

- **Architettura component-based (Front-end)**: il front-end è strutturato secondo il paradigma a componenti di React, in cui ogni elemento dell'interfaccia è un componente riutilizzabile e auto-contenuto. L'organizzazione delle directory riflette questa filosofia:
  - `pages/`: componenti di alto livello che rappresentano le viste dell'applicazione (Kanban Board, Progetti, Report, ecc.)
  - `components/`: componenti riutilizzabili organizzati per dominio funzionale (`issues/`, `projects/`, `layout/`, `common/`)
  - `hooks/`: custom hooks per incapsulare logica riutilizzabile (drag & drop, responsive detection, theming)
  - `context/`: gestione dello stato globale tramite React Context API per autenticazione, progetto corrente e issue
  - `api/`: livello centralizzato per le chiamate HTTP tramite Axios, con interceptor per la gestione automatica dei token JWT
  - `types/`: definizioni TypeScript per la type safety end-to-end
- **Comunicazione REST stateless**: l'interazione tra front-end e back-end avviene tramite API RESTful stateless. Ogni richiesta contiene tutte le informazioni necessarie per essere elaborata (incluso il token JWT di autenticazione), senza che il server mantenga stato di sessione. Questo approccio favorisce la scalabilità orizzontale del back-end.

2.1.2 Motivazioni architetturali

La scelta di un'architettura three-tier con separazione netta tra i componenti è motivata da diverse considerazioni:

- **Manutenibilità**: la stratificazione del codice consente di modificare un livello (ad esempio la logica di business) senza impattare gli altri (presentazione o persistenza), in accordo con il requisito REQ-NFR-05.
- **Testabilità**: la separazione in livelli facilita il testing unitario di ciascuno strato in isolamento, grazie alla possibilità di sostituire le dipendenze con mock o stub.
- **Riusabilità**: l'organizzazione a componenti (sia lato back-end con i Service che lato front-end con i componenti React) favorisce il riuso della logica in diversi contesti applicativi.
- **Sicurezza**: la centralizzazione dell'autenticazione e dell'autorizzazione nel livello back-end, tramite Spring Security, garantisce che le regole di accesso siano applicate uniformemente a tutte le operazioni, indipendentemente dal client che le invoca.
- **Evoluzione indipendente**: l'utilizzo di DTO e API REST come contratto tra front-end e back-end consente ai due componenti di evolvere in modo indipendente, purché il contratto venga rispettato.